Lớp: **IT003.R17**

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Cao Thị Thanh Trúc-25521945

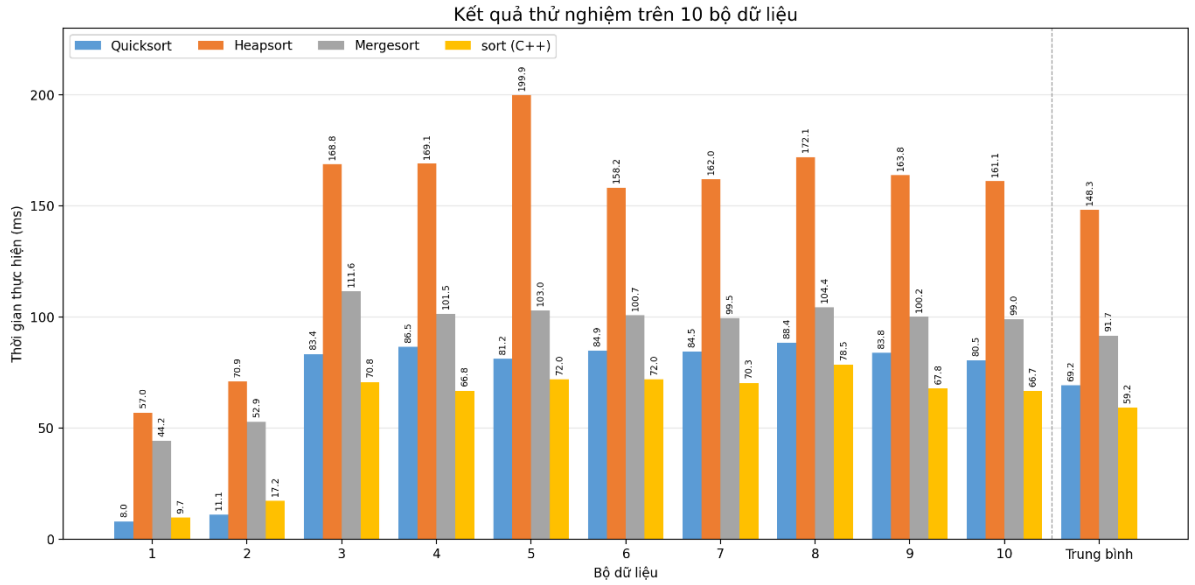
Nội dung báo cáo: Báo cáo kết quả thực nghiệm các thuật toán sắp xếp

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	7.97	56.97	44.16	9.73
2	11.11	70.91	52.85	17.22
3	83.38	168.79	111.62	70.79
4	86.53	169.06	101.51	66.79
5	81.17	199.93	102.97	71.98
6	84.90	158.23	100.73	71.97
7	84.47	162.03	99.54	70.35
8	88.41	172.07	104.40	78.49
9	83.84	163.83	100.15	67.78
10	80.51	161.12	98.95	66.71
Trung bình	69.23	148.29	91.69	59.18

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

1. **Trên dữ liệu ngẫu nhiên (Data 3 đến 10):** thứ tự từ nhanh đến chậm luôn là sort của C++ (khoảng 71 ms), Quicksort (khoảng 84 ms), Mergesort (khoảng 103 ms) và Heapsort (khoảng 169 ms). Thời gian của mỗi thuật toán khá ổn định giữa các dãy, cho thấy thời gian chạy chủ yếu phụ thuộc vào kích thước dữ liệu.

2. **Trên dữ liệu đã có thứ tự (Data 1 tăng dần, Data 2 giảm dần):** cả bốn thuật toán đều chạy nhanh hơn rõ rệt so với dữ liệu ngẫu nhiên. Quicksort nhanh nhất (7,97 ms và 11,11 ms), thậm chí nhanh hơn std::sort (9,73 ms và 17,22 ms), nhờ pivot median-of-three chia mảng cân bằng nên không rơi vào trường hợp xấu nhất $O(n^2)$, đồng thời CPU dự đoán rẽ nhánh tốt hơn. Mergesort và Heapsort chỉ nhanh hơn khoảng 2 đến 3 lần vì vẫn phải sao chép qua mảng phụ (Mergesort) hoặc xây dựng và vun đống (Heapsort) bất kể thứ tự đầu vào.
3. **So sánh chung (thời gian trung bình):** sort C++ (59,18 ms) < Quicksort (69,23 ms) < Mergesort (91,69 ms) < Heapsort (148,29 ms). So với std::sort, Quicksort chậm hơn khoảng 17%, Mergesort khoảng 55% và Heapsort khoảng 2,5 lần.
4. **Giải thích:** cả bốn thuật toán đều có độ phức tạp trung bình O(n log n) nhưng khác nhau về hằng số và cách truy cập bộ nhớ.
    - std::sort là Introsort (Quicksort kết hợp Heapsort và Insertion sort) được tối ưu kỹ nên nhanh nhất trên dữ liệu ngẫu nhiên.
    - Quicksort sắp xếp tại chỗ, duyệt mảng tuần tự nên tận dụng tốt bộ nhớ đệm (cache).
    - Mergesort chậm hơn do phải sao chép qua mảng phụ và tốn thêm bộ nhớ O(n).
    - Heapsort chậm nhất do truy cập phần tử cha/con nhảy cách xa nhau (kém thân thiện với cache) và cần nhiều phép so sánh khi vun đống.
5. **Hạn chế:** mỗi thuật toán chỉ chạy một lần trên mỗi bộ dữ liệu nên có thể chịu ảnh hưởng của các tiến trình nền (thể hiện qua mức dao động của Heapsort từ 158 đến 200 ms).

III. Thông tin chi tiết – link github, trong repo gibub cần có

https://github.com/ttruc1521/IT003-Sorting.git